

Krypto Terra: Detección y Prevención de Exposición de Secretos en Infrastructure as Code

Mariana Malagón Tochoy, Sergio Alejandro Idarraga Torres, Jacobo Diaz Alvarado. Allan Steef Contreras

Abstract—El uso de Infrastructure as Code (IaC) ha transformado la manera en que las organizaciones gestionan y despliegan su infraestructura, permitiendo automatización, escalabilidad y reproducibilidad. Sin embargo, este enfoque ha introducido un riesgo crítico frecuentemente subestimado: la exposición de secretos como credenciales, tokens y contraseñas dentro de los repositorios de código. Este trabajo presenta Kryptos Terra, un enfoque de seguridad basado en tres capas para prevenir, detectar y eliminar la exposición de secretos en entornos IaC.

Index Terms—Infrastructure as Code, seguridad, secretos, DevSecOps, CI/CD, Git

I. INTRODUCCIÓN

Infrastructure as Code (IaC) ha revolucionado la gestión de infraestructuras en entornos modernos, permitiendo a las organizaciones definir, versionar y desplegar recursos mediante código. Sin embargo, esta misma capacidad de versionamiento introduce nuevos riesgos que no existían en modelos tradicionales de administración de infraestructura.

Uno de los problemas más críticos en este contexto es la exposición de secretos dentro del código de infraestructura. En la práctica, es común que desarrolladores incluyan credenciales, tokens o contraseñas directamente en archivos de configuración para agilizar pruebas o despliegues iniciales. Aunque estos valores puedan ser eliminados en commits posteriores, el modelo de historial inmutable de Git garantiza que dichas credenciales permanezcan almacenadas indefinidamente en el repositorio.

Diversos incidentes reales han demostrado la gravedad de este problema. Organizaciones de alto perfil han sufrido filtraciones de credenciales debido a prácticas inadecuadas en la gestión de código, lo que ha derivado en accesos no autorizados, exposición de datos y pérdidas significativas.

En respuesta a esta problemática, este trabajo propone Kryptos Terra, un enfoque basado en múltiples capas de seguridad que aborda la exposición de secretos desde distintas etapas del ciclo de desarrollo. La propuesta combina prevención temprana en el entorno local, detección automatizada en pipelines de integración continua y una solución estructural basada en la gestión centralizada de secretos.

A través de un caso de estudio representativo, se analiza el comportamiento de los secretos en repositorios de IaC y se evalúa la efectividad del sistema propuesto.

II. PLANTEAMIENTO DEL PROBLEMA

En entornos de IaC, los desarrolladores frecuentemente realizan commits con credenciales, tokens y contraseñas directamente en los archivos de configuración de Terraform. El

problema crítico es que aunque se "borren" con un nuevo commit, los secretos permanecen eternamente en el historial de Git, accesibles para cualquiera con acceso al repositorio. Cada vez que ese código se despliega, las credenciales viajan con él hacia múltiples entornos sin que nadie lo note.

Ciclo de vida del secreto

- 1) **Nacimiento:** un desarrollador escribe credenciales reales en un archivo de Terraform para "probar rápido" y realiza un commit sin ser conciente del riesgo.
- 2) **Replica:** ese commit se sube al repositorio, viaja a las ramas, se clona en otras máquinas y se incluye en backups.
- 3) **Aparente eliminación:** alguien nota que hay credenciales expuestas, así que las elimina y hace un nuevo commit pensando que resolvió todo.
- 4) **Vida en Git:** el secreto existe para siempre en el historial de Git. Cualquier persona con acceso puede recuperarlo volviendo al commit anterior. Las organizaciones no lo saben hasta que alguien lo explota.

III. REVISIÓN DE LITERATURA

Casos reales documentados que ilustran esto: Toyota en 2023 tuvo una clave de acceso a infraestructura cloud expuesta en un repositorio público durante 5 años. Samsung filtró código fuente y claves privadas por empleados que subieron archivos a GitHub. Uber sufrió el robo de credenciales de AWS desde un repositorio interno, lo que permitió exfiltrar datos de 57 millones de usuarios.

IV. MARCO TEÓRICO

A. Infrastructure as Code (IaC)

Infrastructure as Code (IaC) es un paradigma que permite gestionar y aprovisionar infraestructura mediante archivos de configuración versionados, en lugar de procesos manuales. Herramientas como Terraform permiten definir recursos como servidores, bases de datos y redes utilizando código declarativo, facilitando la automatización, reproducibilidad y escalabilidad de los entornos.

Este enfoque ha transformado la forma en que las organizaciones despliegan sistemas, integrándose de manera natural con prácticas de integración y entrega continua (CI/CD). Sin embargo, al tratar la infraestructura como código, también se heredan los mismos riesgos de seguridad asociados al desarrollo de software, particularmente en lo relacionado con la gestión de información sensible.

B. Git y su historial inmutable

Git es un sistema de control de versiones distribuido que almacena el historial completo de cambios realizados en un repositorio. Una de sus características fundamentales es la inmutabilidad del historial, lo que implica que cada commit registrado permanece accesible de forma permanente, incluso si versiones posteriores eliminan o modifican su contenido.

Este comportamiento es especialmente problemático en el contexto de la seguridad, ya que cualquier información sensible incluida en un commit puede ser recuperada fácilmente accediendo a versiones anteriores del repositorio.

C. Secretos en software

En el desarrollo de software, los secretos corresponden a cualquier tipo de información sensible utilizada para autenticación o autorización. Entre los ejemplos más comunes se incluyen claves de acceso (API keys), tokens de autenticación, credenciales de servicios en la nube, contraseñas de bases de datos y claves privadas criptográficas.

D. DevSecOps

DevSecOps significa desarrollo, seguridad y operaciones. Es una extensión de la práctica de DevOps. Es la práctica de integrar las pruebas de seguridad en cada etapa del desarrollo de software. Sus herramientas y procesos promueven la colaboración entre los desarrolladores, los especialistas en seguridad y los equipos de operaciones para crear un software que sea eficiente y seguro. Es una cultura que hace de la seguridad una responsabilidad compartida para todos los que crean el software.

Mientras que en modelos tradicionales la seguridad se evaluaba al final del proceso, DevSecOps propone que esta se integre desde las primeras etapas, incluyendo el diseño, desarrollo, integración y despliegue. Esto permite detectar vulnerabilidades de forma temprana, reduciendo costos y evitando que errores críticos lleguen a producción.

E. Gestión de secretos

La gestión de secretos consiste en la protección de credenciales sensibles como certificados, claves, contraseñas y tokens utilizados por entidades no humanas, como aplicaciones, servidores y cargas de trabajo.

Este tipo de cuentas privilegiadas representan un objetivo atractivo para atacantes, ya que permiten acceder a sistemas sin necesidad de intervención humana directa. Cuando estas credenciales son comprometidas, pueden ser utilizadas para escalar privilegios, extraer información o afectar servicios críticos.

De hecho, el secuestro de cuentas válidas es actualmente uno de los vectores de ataque más comunes. Según el informe IBM X-Force Threat Intelligence Index, este tipo de incidentes representa aproximadamente el 30% de los casos recientes atendidos, lo que evidencia la importancia de proteger adecuadamente este tipo de información.

F. Análisis estático de seguridad

Las herramientas de análisis de código estático ayudan a los desarrolladores a cumplir las pautas de codificación, la revisión de los lenguajes de programación y los estándares establecidos en el equipo de desarrollo, garantizando una base de código consistente y coherente. Permiten identificar y rectificar problemas que podrían pasarse por alto durante las revisiones manuales del código y, en última instancia, mejoran la calidad general del software.

Además de mejorar la calidad, este tipo de análisis cumple un rol fundamental en la seguridad, ya que permite detectar vulnerabilidades sin necesidad de ejecutar el código. Estas herramientas analizan patrones específicos asociados a riesgos, como credenciales expuestas, configuraciones inseguras o uso indebido de recursos.

V. PROPUESTA TÉCNICA

El sistema propuesto se estructura en tres capas de protección complementarias que actúan en distintos momentos del ciclo de desarrollo. La idea principal es que ninguna capa por sí sola es suficiente; un desarrollador puede omitir controles locales, un pipeline puede no cubrir todos los escenarios y una mala gestión de secretos puede anular cualquier medida previa.

A. Pre-commit Hook

La primera línea de defensa opera directamente en la máquina del desarrollador antes de que cualquier información sensible toque el historial de Git. Se configura Gitleaks como un hook de pre-commit que se ejecuta automáticamente cada vez que el desarrollador intenta registrar un cambio. El hook analiza los archivos staged buscando patrones conocidos de credenciales: claves de AWS, tokens de GitHub, contraseñas en texto plano, claves privadas SSH, cadenas de conexión a bases de datos y más de 150 tipos de secretos adicionales que Gitleaks reconoce por defecto. Si el análisis no encuentra nada, el commit procede normalmente sin que el desarrollador note ninguna diferencia en su flujo de trabajo. Si encuentra un patrón sospechoso, el commit es rechazado inmediatamente con un reporte que indica el archivo, la línea y el tipo de secreto detectado. El desarrollador debe resolver el problema antes de poder continuar. Esta capa es la más económica en términos de impacto porque actúa antes de que el daño ocurra, pero tiene una limitación importante: depende de que el desarrollador tenga el hook instalado y activo, lo cual no siempre está garantizado en equipos grandes o con rotación de personal.

B. Pipeline CI/CD

La segunda capa opera de forma independiente a la máquina del desarrollador y cubre los casos que la primera capa no puede garantizar. Se implementa un workflow en GitHub Actions que se dispara automáticamente en cada push y en cada pull request, sin importar la rama. Este workflow ejecuta TruffleHog sobre el historial completo del repositorio, no solo sobre el último commit, lo cual es crítico porque permite

detectar secretos que fueron introducidos y luego "borrados" en commits anteriores. Cuando TruffleHog encuentra un hallazgo, el workflow falla y el merge queda bloqueado automáticamente. El reporte queda visible en la interfaz de GitHub con el detalle de cada hallazgo, incluyendo el commit de origen, el archivo y el tipo de secreto. Esto crea un registro auditable de cada incidente de seguridad detectado y notifica al equipo sin necesidad de intervención manual. A diferencia del hook local, esta capa no puede ser saltada por un desarrollador individual porque opera en la infraestructura del repositorio y es administrada por el equipo de seguridad o el líder técnico.

C. Gestión Centralizada de Secretos

Las dos capas anteriores son mecanismos de detección y bloqueo, pero no resuelven el problema de fondo: que los secretos no deberían estar en el código de Terraform en ningún momento. La tercera capa aborda esto de forma estructural mediante HashiCorp Vault, un sistema de gestión de secretos que actúa como fuente única de verdad para todas las credenciales de la infraestructura. En lugar de escribir valores reales en los archivos de Terraform, los desarrolladores definen referencias a rutas dentro de Vault. En tiempo de ejecución, Terraform se autentica con Vault, recupera el valor del secreto y lo usa para el despliegue sin que ese valor jamás quede escrito en ningún archivo ni en ningún log del pipeline. El código de infraestructura puede ser completamente público porque no contiene información sensible: solo contiene instrucciones sobre dónde encontrar los secretos, no los secretos en sí mismos. Vault además ofrece control de acceso granular, rotación automática de credenciales y registro de auditoría de cada acceso, características que las otras dos capas no proveen.

VI. IMPLEMENTACIÓN

La implementación de Kryptos Terra se estructuró en tres fases secuenciales, cada una orientada a transformar progresivamente un entorno vulnerable en una infraestructura resiliente y segura.

A. Fase 1: Escenario Vulnerable y Análisis Forense

Se creó un repositorio de referencia que contenía archivos de configuración de Terraform con credenciales expuestas de forma intencional (simuladas) para servicios de AWS y conexiones a bases de datos. Tras realizar un commit inicial, dichas credenciales fueron eliminadas manualmente en un segundo commit, simulando un intento común de remediación.

Con el fin de evaluar la efectividad de esta acción, se ejecutó el comando `gitleaks detect -log-opts="-all"`. Los resultados evidenciaron que la información sensible permanecía accesible en la base de datos de objetos de Git, confirmando que la eliminación superficial de secretos no los remueve del historial del repositorio. Esta fase pone de manifiesto la persistencia de credenciales filtradas y la necesidad de aplicar estrategias de remediación más profundas.

B. Fase 2: Capas de Prevención y Detección con Gitleaks

Se estandarizó el uso de Gitleaks como herramienta principal de detección de secretos en dos capas complementarias:

Capa Local: Se configuró un pre-commit hook para prevenir la inclusión de información sensible durante el proceso de registro de cambios. Este mecanismo bloquea automáticamente los commits cuando se detectan patrones asociados a claves API, tokens o credenciales. **Capa de CI/CD:** Se integró un workflow de GitHub Actions para ejecutar escaneos automáticos en cada push y pull request. La configuración del flujo incluye `fetch-depth: 0`, garantizando un análisis completo del historial del repositorio. Asimismo, se habilitó el parámetro `-redact` con el fin de evitar la exposición de datos sensibles en los registros del sistema de integración continua.

Este enfoque multicapa fortalece tanto la detección temprana como el monitoreo continuo de posibles filtraciones de secretos.

C. Fase 3: Remediación Estructural con HashiCorp Vault

Para abordar la causa raíz de la exposición de credenciales, se desplegó una instancia de HashiCorp Vault como solución centralizada de gestión de secretos. Posteriormente, se refactorizó la base de código de Terraform para eliminar valores sensibles definidos de forma estática (hardcoded).

En particular, se implementó el bloque `data "vault-kv-secret-v2"` para permitir la recuperación dinámica de credenciales, tales como `access-key` y `secret-key`, directamente desde el motor de secretos tipo Key-Value (KV) de Vault en tiempo de ejecución. Este enfoque garantiza que la información sensible no se almacene en el código fuente, reduciendo significativamente el riesgo de exposición.

En conjunto, esta fase establece una arquitectura *secure-by-design*, basada en la externalización de secretos y el acceso dinámico a credenciales.

VII. RESULTADOS Y VALIDACIÓN

Las pruebas de validación confirmaron que el enfoque multicapa mitiga eficazmente los riesgos asociados a la gestión inadecuada de secretos.

TABLE I
COMPARATIVA DE SEGURIDAD: ANTES VS. DESPUÉS

Criterio de Evaluación	Estado Inicial	Kryptos Terra
Secretos en archivos .tf	Presentes	Eliminados
Secretos en historial Git	Expuestos	Detectados/Bloqueados
Gestión de credenciales	Manual/Insegura	Centralizada (Vault)
Falla de pipeline ante hallazgos	No	Sí (Bloqueo de merge)

A. Análisis de Efectividad

La efectividad del enfoque propuesto se sustenta en las siguientes observaciones:

- 1) **Prevención en el origen:** El *pre-commit hook* local evitó el 100% de los intentos de *commit* que contenían nuevos secretos durante la fase de desarrollo, evidenciando una protección proactiva robusta.

- 2) **Visibilidad histórica:** El pipeline de CI/CD detectó de manera exitosa los secretos residuales del escenario vulnerable (denominados “secretos zombie”), impidiendo su integración en ramas protegidas y garantizando la integridad del repositorio.
- 3) **Abstracción de secretos:** La integración con HashiCorp Vault permitió que el repositorio de infraestructura pudiera considerarse, en la práctica, como público sin comprometer la seguridad de la nube. Esto se logra mediante la eliminación de credenciales embebidas y su sustitución por referencias lógicas a rutas de secretos gestionadas externamente.

En conjunto, estos resultados demuestran que la combinación de controles preventivos, escaneo continuo y gestión segura de secretos mejora significativamente la postura de seguridad en entornos de infraestructura como código (*Infrastructure as Code*, IaC).

VIII. CONCLUSIONES

Kryptos Terra evidencia que la seguridad en entornos de *Infrastructure as Code* (IaC) debe abordarse como un modelo integral basado en tres dimensiones fundamentales: prevención, detección y gestión de secretos. La implementación coordinada de estas capas permite reducir significativamente la superficie de ataque y fortalecer la postura de seguridad de la infraestructura.

- **Defensa en profundidad:** La detección de secretos, aunque necesaria, no es suficiente por sí sola. Es indispensable complementar estos mecanismos con estrategias de eliminación y externalización de credenciales. En este contexto, herramientas como Gitleaks funcionan como una capa de control preventivo y de monitoreo, mientras que soluciones como HashiCorp Vault constituyen el componente central para una gestión segura y sostenible de secretos.
- **Automatización y cultura de seguridad:** La integración de controles de seguridad en el ciclo de vida de desarrollo, particularmente en los flujos de trabajo de Git y CI/CD, reduce la dependencia de intervenciones manuales y minimiza errores humanos. Este enfoque promueve una cultura de seguridad por defecto, en la cual las buenas prácticas son inherentes al proceso de desarrollo.
- **Gestión del riesgo histórico:** Un secreto expuesto en el historial de un repositorio debe considerarse comprometido de forma permanente. En este sentido, Kryptos Terra no solo previene futuras filtraciones, sino que también enfatiza la necesidad de auditar y remediar exposiciones pasadas. Este enfoque garantiza una infraestructura más confiable, alineada con principios de seguridad proactiva y cumplimiento.

En conjunto, los resultados obtenidos demuestran que la adopción de un enfoque multicapa, apoyado en herramientas automatizadas y en la centralización de secretos, constituye una estrategia efectiva para asegurar entornos IaC modernos.

AGRADECIMIENTOS

Los autores agradecen a la Universidad Escuela Colombiana de Ingeniería Julio Garavito y al profesor Fabian Eduardo Sierra por su guía durante el desarrollo de este proyecto.

REFERENCES

- [1] Amazon Web Services, “¿Qué es DevSecOps?,” [En línea]. Disponible en: <https://aws.amazon.com/es/what-is/devsecops/>. [Accedido: 08-abr-2026].
- [2] IBM, “Gestión de secretos,” [En línea]. Disponible en: <https://www.ibm.com/mx-es/think/topics/secrets-management>. [Accedido: 08-abr-2026].
- [3] GitGuardian, “State of Secrets Sprawl Report,” [En línea]. Disponible en: <https://www.gitguardian.com>. [Accedido: 08-abr-2026].
- [4] HashiCorp, “Vault Documentation,” [En línea]. Disponible en: <https://developer.hashicorp.com/vault>. [Accedido: 08-abr-2026].